- Testing all the business logic of the provider and side effects of a particular functionality (should be covered by a functional test instead from the provider)
- Testing the UI's components such as its visual look or accessibility requirements
- Testing other requirements such as performance or security
- Using third-party APIs where you don't have much control over how contract tests can be added to their pipelines
- Using public APIs where you don't know who is consuming your data

We'll explain in chapter 11, however, how you can still use contract testing if you're working with third-party or public APIs via an approach called bi-directional contract testing.

1.4 How contract testing works on a high level

There are a few approaches to adopting contract testing, all of which we'll discuss in more detail in later chapters. However, as part of this introduction, we've only described how contract testing works at a high level, which covers the original consumer-driven contract testing approach in which the consumer (see chapter 3, section 3.1, for more details on consumers) creates the contract and shares with the data providers.

Earlier in the chapter, we described how traditional API integration testing works. We highly recommend understanding the fundamentals of API testing before learning how contract testing works. If you haven't already, we suggest reading about RESTful APIs and API testing, especially, Mark Winteringham's book, *Testing Web APIs* (www.manning.com/books/testing-web-apis). Contract testing can be seen as a form of integration testing but with some differences, such as an asynchronous unit test.

Imagine you're buying a house (in the UK). When someone makes an offer on a property in the UK, the buyer and seller must communicate to iron out the details of the contract; however, both sides have their own copies until later in the process. Think of this in software terms as distributed